Grounded Knowledge Search

AI Engineer Assignment – Developer Implementation Plan

Project: Grounded Knowledge Search
Role: AI Engineer
Estimated Development Time: 3 Focused Days
Primary Objective: Build a reliable, access-controlled, citation-based retrieval and question-answering system for financial filings.

---

1. Project Overview

1.1 Objective

Develop a local RAG-based knowledge search system that allows equity research analysts to search financial company filings and receive accurate, evidence-based answers.

The system must:

- Ingest all 119 financial PDF filings.
- Extract text and financial tables while preserving page numbers.
- Convert extracted content into searchable chunks.
- Generate embeddings and store them in a vector database.
- Retrieve relevant information based on user questions.
- Enforce company-level access control based on analyst roles.
- Generate answers with document and page citations.
- Refuse to answer when sufficient evidence is unavailable.
- Demonstrate measurable retrieval improvement over a baseline.
- Evaluate retrieval and answer quality using at least 30 questions.

1.2 Provided Dataset

The assignment provides three main resources:

Resource	Description	Status
`pdfs/`	119 financial PDF filings from 12 companies	Provided
`manifest.csv`	Document metadata including company, sector, desk, document type and period	Provided
`dev_questions.jsonl`	20 questions with expected answers, document IDs and evidence pages	Provided

Dataset Characteristics

Companies: 12 US-listed companies.

Coverage: 2015–2024.

Document types:

- 10-K: Annual reports
- 10-Q: Quarterly reports
- 8-K: Current reports
- Earnings: Earnings releases

Important observation:

Most companies have multiple filings across different years. The same financial metrics may appear in several documents with different values.

Therefore, retrieval must consider company, reporting period, document type and page-level evidence rather than relying only on semantic similarity.

1.3 Analyst Roles and Access Permissions

Role	Accessible Companies
`consumer`	PepsiCo, Best Buy, MGM Resorts, Ulta Beauty
`health_industrials`	Johnson & Johnson, Pfizer, 3M, Boeing
`tech_telecom`	AMD, Adobe, Corning, Verizon
`lead`	All 12 companies

Access control must be enforced inside the retrieval process using metadata filters.

---

2. Technology Stack

Use lightweight, locally runnable technologies.

Component	Recommended Technology	Purpose
Programming Language	Python	Core implementation
PDF Parsing	PyMuPDF	Page-wise text extraction
Table Extraction	pdfplumber	Financial table extraction
Embedding Model	Sentence Transformers	Generate vector embeddings
Embedding Model	`all-MiniLM-L6-v2`	Initial baseline model
Vector Database	ChromaDB	Store and retrieve embeddings
Keyword Retrieval	BM25 / rank_bm25	Hybrid retrieval experiment
LLM	Ollama with a small local model	Grounded answer generation
CLI	argparse	Implement required commands
Evaluation	Python, Pandas	Retrieval and answer evaluation
Configuration	python-dotenv	Environment-based configuration
Testing	pytest	Functional and access-control tests

Implementation rule: Start with the recommended baseline stack. Do not introduce additional libraries unless an experiment demonstrates a real benefit.

The LLM and embedding model should be configurable independently.

---

3. Development Roadmap

PHASE 1 — Dataset Inspection and Initial Analysis

Priority: Critical
Estimated Time: 1–2 hours

Objective

Understand the actual structure and quality of the provided dataset before implementing the ingestion pipeline.

Task 1.1 — Inspect Dataset Structure

Check:

- Whether all 119 PDFs are available.
- Whether `manifest.csv` contains 119 corresponding records.
- Whether every manifest document has a matching PDF.
- Whether any PDFs are missing or duplicated.
- Whether document IDs match the PDF filenames.
- Whether the JSONL file contains all 20 development questions.

Create a dataset inspection script.

Suggested file:

`src/analysis/inspect_corpus.py`

Task 1.2 — Analyze Manifest

Read the manifest and inspect:

- Total documents.
- Unique companies.
- Documents per company.
- Documents per desk.
- Distribution of document types.
- Distribution of reporting periods.

Important: Verify the actual file delimiter. The provided manifest example appears tab-separated, despite being named `manifest.csv`.

Also preserve exact `doc_id` values. Do not normalize IDs blindly because some entries use different capitalization.

Task 1.3 — Inspect PDF Extraction Quality

Select representative PDFs:

- One 10-K.
- One 10-Q.
- One 8-K.
- One earnings release.

Extract several pages from each.

Inspect:

- Text readability.
- Table extraction.
- Financial figures.
- Headers and footers.
- Page numbering.
- Missing or incorrectly extracted content.

Pay particular attention to balance sheets, income statements and cash flow statements.

Expected Deliverables

Create:

`CORPUS_ANALYSIS.md`

Include:

1. Dataset statistics.
2. Company and document distribution.
3. PDF extraction observations.
4. Table extraction observations.
5. Data quality issues.
6. Initial design decisions based on observations.

Completion Criteria

- All 119 PDFs verified against the manifest.
- All 20 development questions loaded successfully.
- Manifest structure understood.
- Four representative PDF types inspected.
- Text and table extraction quality evaluated.
- Initial findings documented.

Expected Output: A documented understanding of the dataset and a justified choice of PDF extraction strategy.

PHASE 2 — Project Setup and Configuration

Estimated Time: 30–45 minutes

Objective

Create a clean, modular Python project that can run on another machine without paid services.

Task 2.1 — Create Project Structure

```
None
grounded-knowledge-search/
│
├── pdfs/
├── eval/
│   ├── questions.jsonl
│   ├── run_eval.py
│   └── metrics.py
│
├── src/
│   ├── ingestion/
│   │   ├── parser.py
│   │   ├── table_extractor.py
│   │   ├── chunker.py
│   │   └── ingest.py
│   │
│   ├── retrieval/
│   │   ├── embeddings.py
│   │   ├── vector_store.py
│   │   ├── search.py
│   │   └── hybrid_search.py
│   │
```

```
│   ├── generation/
│   │   ├── llm.py
│   │   └── answer.py
│   │
│   ├── analysis/
│   │   └── inspect_corpus.py
│   │
│   ├── access_control.py
│   └── config.py
│
├── data/
│   ├── extracted/
│   └── chroma_db/
│
├── docs/
│
├── main.py
├── requirements.txt
├── .env.example
├── .gitignore
│
├── README.md
├── CORPUS_ANALYSIS.md
├── DECISIONS.md
├── EXPERIMENTS.md
└── AI_NOTES.md
```

Task 2.2 — Configuration

Create configurable settings for:

- Embedding model.
- LLM provider.
- LLM model.
- Vector database location.
- Chunk size.
- Chunk overlap.
- Retrieval top-k.

- Maximum context size.

Use environment variables where necessary.

Expected Output

A working Python environment with a modular project structure.

Completion Criteria

- Dependencies installed.
- Configuration module created.
- Environment variables configured.
- Git initialized.
- `.env` excluded from Git.
- Initial Git commit created.

PHASE 3 — PDF Ingestion Pipeline

Estimated Time: 4–5 hours

Objective

Extract searchable information from all 119 PDF filings while preserving document identity and page numbers.

This phase is particularly important because financial answers frequently appear inside tables.

Task 3.1 — PDF Text Extraction

Implement:

`src/ingestion/parser.py`

Responsibilities:

1. Read a PDF.
2. Process it page by page.
3. Extract text.
4. Preserve the original PDF page number.
5. Handle empty pages.

6. Handle extraction errors.
7. Return structured page-level content.

Suggested output:

```
None
{
    "doc_id": "3M_2018_10K",
    "page": 60,
    "text": "Capital expenditures for 2018...",
    "content_type": "text"
}
```

Important: Page numbering must be 1-based, matching the assignment's citation convention.

Do not use a continuous document-level text extraction approach that loses page boundaries.

Task 3.2 — Financial Table Extraction

Implement:

`src/ingestion/table_extractor.py`

Responsibilities:

- Detect and extract tables.
- Preserve row and column relationships.
- Preserve financial labels.
- Preserve associated values and units.
- Associate each table with its original PDF page.
- Convert extracted tables into searchable text.

Example:

Original table:

Financial Metric	2018
Accounts Payable	303
Accounts Receivable	450
Inventory	120

Searchable representation:

```
None
Financial Statement: Balance Sheet
Fiscal Year: 2018
Accounts Payable: 303 million USD
Accounts Receivable: 450 million USD
Inventory: 120 million USD
```

The example is illustrative. Actual values and units must come from the PDF.

Do not discard the original table structure during conversion.

Task 3.3 — Combine Extracted Content

Combine normal text and table content into a consistent page-level representation.

Every record must preserve:

- `doc_id`
- `company`
- `sector`
- `desk`
- `doc_type`
- `period`
- `page`
- `text`
- `content_type`

Metadata must come from the manifest rather than being guessed from PDF content.

Task 3.4 — Save Intermediate Extraction Results

Save extracted content locally.

Suggested structure:

```
None
data/
```

```
└── extracted/
    ├── 3M_2018_10K.json
    ├── AMD_2022_10K.json
    ├── PFIZER_2022_10K.json
    └── ...
```

This makes it possible to debug parsing without repeatedly processing every PDF.

Expected Output

Page-level extracted JSON files for all 119 filings.

Example:

None

```
{
  "doc_id": "3M_2018_10K",
  "company": "3M",
  "sector": "Industrials",
  "desk": "health_industrials",
  "doc_type": "10k",
  "period": "2018",
  "pages": [
    {
      "page": 60,
      "content": [
        {
          "type": "text",
          "text": "..."
        },
        {
          "type": "table",
          "text": "..."
        }
      ]
    }
  ]
```

```
}
```

Completion Criteria

- All 119 PDFs processed.
- Page numbers preserved.
- Tables extracted.
- Text and table content distinguishable.
- Manifest metadata attached.
- Extraction errors logged.
- Sample outputs manually verified against original PDFs.

Important: If table extraction fails on a particular document, record the issue instead of silently treating the document as successfully processed.

PHASE 4 — Chunking and Embedding

Estimated Time: 2–3 hours

Objective

Convert extracted content into searchable chunks and generate embeddings.

Task 4.1 — Implement Page-Aware Chunking

Create:

`src/ingestion/chunker.py`

Start with a simple fixed-size chunking strategy.

Initial experimental configuration:

- Chunk size: approximately 500 tokens.
- Overlap: approximately 100 tokens.

These are starting values, not final decisions.

Requirements:

1. Split long page content into manageable chunks.
2. Preserve page numbers.
3. Preserve document metadata.
4. Avoid losing financial table rows.
5. Keep table labels and values together whenever possible.
6. Generate deterministic chunk IDs.

For short pages, keep the content together rather than unnecessarily splitting it.

For tables, use a separate table-aware chunking strategy if ordinary text chunking damages row relationships.

Task 4.2 — Generate Embeddings

Create:

`src/retrieval/embeddings.py`

Use:

`sentence-transformers/all-MiniLM-L6-v2`

Generate an embedding for each chunk.

The embedding model must be configurable through environment settings.

Task 4.3 — Generate Stable Chunk IDs

Chunk IDs should be deterministic.

A suitable approach is hashing:

- Document ID.
- Page number.
- Chunk content.
- Chunk position.

This helps prevent duplicate records during ingestion.

Task 4.4 — Store Chunks in ChromaDB

Create:

`src/retrieval/vector_store.py`

Store:

- Chunk ID.
- Chunk text.
- Embedding.
- Document metadata.

Example:

```
None
{
    "doc_id": "3M_2018_10K",
    "company": "3M",
    "sector": "Industrials",
    "desk": "health_industrials",
    "doc_type": "10k",
    "period": 2018,
    "page": 60
}
```

Task 4.5 — Implement Idempotent Ingestion

Re-ingesting the same unchanged filing must not create duplicate chunks.

Also handle changed filings:

1. Calculate a file hash.
2. Compare it with the previously indexed hash.
3. Skip unchanged files.
4. Remove or replace old chunks if a filing has changed.
5. Insert the newly generated chunks.

Expected Output

A populated ChromaDB collection containing searchable chunks from the corpus.

Completion Criteria

- All extracted content chunked.
- Every chunk has a valid page number.
- Embeddings generated successfully.
- Metadata stored correctly.

- Stable chunk IDs generated.
- Re-ingestion does not create duplicates.
- Changed filings replace their old indexed content.

PHASE 5 — Baseline Retrieval System

Estimated Time: 2 hours

Objective

Implement the simplest functional retrieval system and measure its performance before introducing improvements.

Task 5.1 — Implement Vector Search

Create:

`src/retrieval/search.py`

Implement semantic similarity search using ChromaDB.

Input:

- Question.
- Role.
- Top-k.

Process:

1. Receive the question.
2. Generate its embedding.
3. Apply the role-based metadata filter.
4. Retrieve the top-k matching chunks.
5. Return results in descending relevance order.

Task 5.2 — Implement Access Control

Create:

`src/access_control.py`

Use the manifest's `desk` field as the source of truth.

Role mapping:

```
None
ROLE_DESKS = {
    "consumer": ["consumer"],
    "health_industrials": ["health_industrials"],
    "tech_telecom": ["tech_telecom"],
    "lead": [
        "consumer",
        "health_industrials",
        "tech_telecom"
    ]
}
```

Apply access restrictions directly inside the vector database query.

Do not retrieve unauthorized documents and then attempt to remove them using the LLM prompt.

Test cross-desk access explicitly.

Task 5.3 — Implement Search Output

Required command:

```
None
python main.py search --role consumer --k 5 "What were PepsiCo's
capital expenditures in 2021?"
```

Expected output:

```
None
[
  {
    "doc_id": "PEPSICO_2021_10K",
    "page": 63,
```

```
    "chunk_id": "chunk_abc123",
    "score": 0.89,
    "text": "..."
  }
]
```

The score must have a documented interpretation and consistent sorting.

Task 5.4 — Evaluate Baseline

Run the retrieval system against the provided 20 questions.

Compare:

- Expected document ID.
- Expected evidence pages.
- Retrieved document IDs.
- Retrieved pages.
- Ranking of relevant evidence.

Record the initial metrics in `EXPERIMENTS.md`.

Expected Output

A working baseline search system with recorded retrieval metrics.

Completion Criteria

- Search command works.
- JSON output matches the interface contract.
- Results are sorted by relevance.
- Role filtering works.
- No unauthorized document is returned.
- Baseline evaluated on 20 questions.
- Baseline results documented.

PHASE 6 — Retrieval Improvement Experiments

Estimated Time: 4–5 hours

Objective

Improve retrieval performance through controlled experiments.

This is the most important evaluation area of the assignment.

Do not change several retrieval components simultaneously. Each experiment should test one hypothesis.

Experiment E1 — Baseline

Approach: Fixed-size chunking + vector similarity search.

Record the initial metrics.

Experiment E2 — Chunk Size Optimization

Hypothesis:

The initial chunk size may separate financial figures from their labels or include too much irrelevant content.

Test alternative chunk sizes.

For example:

- 300 tokens.
- 500 tokens.
- 800 tokens.

Keep overlap controlled during this experiment.

Compare Recall@5, MRR and page accuracy.

Choose the configuration that performs best on the evaluation set.

Experiment E3 — Hybrid Search

Hypothesis:

Semantic search may miss exact financial terms, company-specific terminology and numerical disclosures.

Implement BM25 keyword retrieval alongside vector search.

Combine rankings using Reciprocal Rank Fusion (RRF).

Example:

```
None
Question
   |
   |--------------------|
   v                    v
Vector Search       BM25 Search
   |                    |
   |--------------------|
            |
            v
     Rank Fusion
            |
            v
    Final Top-K Results
```

Compare hybrid retrieval against the best vector-only configuration.

Do not assume hybrid search is automatically better. Keep it only if measured results justify it.

Experiment E4 — Query and Metadata Improvements

Test whether structured query information improves retrieval.

Potential changes:

- Extract company name.
- Identify reporting year.
- Identify document type where specified.
- Use company and period metadata filters when the question clearly identifies them.

Important: Apply filters only when supported by the question. Do not unnecessarily exclude relevant filings.

Compare results against the previous best configuration.

Experiment E5 — Table-Aware Retrieval

Hypothesis:

Financial table extraction and chunking may be responsible for missed numerical evidence.

Test:

- Normal text chunks.
- Table-specific chunks.
- Combined text and table retrieval.

Evaluate questions that require:

- Balance sheet figures.
- Income statement values.
- Cash flow values.
- Financial ratios.
- Multiple-year calculations.

Keep this improvement only if it produces measurable gains.

Experiment Tracking

Maintain this table in `EXPERIMENTS.md`:

Experiment	Change	Hypothesis	Recall@5	MRR	Decision
E1	Baseline vector search	Initial benchmark	TBD	TBD	Baseline
E2	Chunk size	Better context boundaries	TBD	TBD	Pending
E3	Hybrid search	Better exact-term retrieval	TBD	TBD	Pending
E4	Query improvements	Better document targeting	TBD	TBD	Pending

E5	Table-aware retrieval	Better financial evidence	TBD	TBD	Pending

Expected Output

A measured comparison of different retrieval approaches and a final selected retrieval configuration.

Completion Criteria

- At least three meaningful retrieval experiments performed.
- Each experiment tests a clear hypothesis.
- Metrics recorded.
- Negative results documented.
- Final retrieval configuration selected based on evidence.

PHASE 7 — Grounded Question Answering

Estimated Time: 3 hours

Objective

Implement the `ask` command using retrieved evidence and a local LLM.

Task 7.1 — LLM Integration

Create:

`src/generation/llm.py`

Use Ollama with a lightweight local model.

Keep the LLM provider configurable so a different free provider can be selected using environment variables.

The system must not require a paid API account.

Task 7.2 — Grounded Answer Generation

Create:

`src/generation/answer.py`

Pass the retrieved evidence to the LLM.

Use a strict system prompt with these instructions:

1. Answer only using the supplied filing evidence.
2. Do not use outside knowledge.
3. Do not invent financial values.
4. Do not mix figures from different reporting years.
5. Cite the document and page supporting each claim.
6. If evidence is insufficient, return `refused`.
7. For calculations, explicitly show the input values and formula.

Task 7.3 — Implement Refusal Logic

Refuse when:

- No relevant evidence is retrieved.
- Retrieved evidence does not answer the question.
- Required financial inputs are missing.
- The question cannot be answered reliably from the available filings.

Do not rely exclusively on the LLM to decide whether evidence is sufficient.

Implement basic evidence validation before generating an answer.

Task 7.4 — Calculation Support

For calculated metrics, include:

- Input figures.
- Reporting periods.
- Formula.
- Final calculated result.
- Citations for the supporting inputs.

Example:

```
None
Revenue Growth =
```

```
((Current Year Revenue - Previous Year Revenue)
 / Previous Year Revenue) * 100
```

For multi-document or multi-page calculations, cite every relevant source.

Task 7.5 — Implement Ask Output

Required command:

```
None
python main.py ask --role consumer "What was PepsiCo's capital
expenditure in 2021?"
```

Expected output:

```
None
{
  "status": "answered",
  "answer": "PepsiCo reported capital expenditures of $4.60 billion
in FY2021.",
  "citations": [
    {
      "doc_id": "PEPSICO_2021_10K",
      "page": 63,
      "chunk_id": "chunk_abc123"
    }
  ]
}
```

Example refusal:

```
None
{
  "status": "refused",
```

```
  "answer": "The available filings do not contain sufficient
evidence to answer this question.",
  "citations": []
}
```

Expected Output

A working grounded Q&A system with valid citations and refusal handling.

Completion Criteria

- Local LLM integration works.
- Answers use retrieved evidence.
- Citations contain valid document IDs and page numbers.
- Unsupported claims are not presented as facts.
- Missing evidence produces refusal.
- Calculations show their inputs and formulas.
- JSON output matches the interface contract.

PHASE 8 — Evaluation Framework

Estimated Time: 3 hours

Objective

Build a repeatable evaluation framework that measures retrieval quality and answer quality independently.

Task 8.1 — Create Evaluation Dataset

Create:

`eval/questions.jsonl`

Start with the 20 provided questions.

Add at least 10 original questions.

The additional questions should cover different types of retrieval:

1. Financial figures.
2. Revenue and expense disclosures.
3. Balance sheet information.
4. Cash flow information.
5. Multi-year comparisons.
6. Company disclosures.
7. Financial calculations.
8. Questions involving multiple documents.
9. Questions with insufficient evidence.
10. Cross-company questions for the lead role.

Use the actual filings to establish the expected answers and evidence pages.

Do not generate expected answers without verifying the source documents.

Suggested format:

```
None
{
  "id": "custom_001",
  "role": "consumer",
  "question": "What was Best Buy's operating cash flow in FY2023?",
  "answer": "...",
  "doc_id": "BESTBUY_2023_10K",
  "pages": [42]
}
```

For questions requiring multiple documents, store multiple expected evidence references.

For unanswerable questions, explicitly mark them as requiring refusal.

Task 8.2 — Implement Retrieval Metrics

Create:

`eval/metrics.py`

Measure:

Recall@K

Checks whether relevant evidence appears within the top K retrieved results.

MRR (Mean Reciprocal Rank)

Measures how highly the first relevant result is ranked.

Page Accuracy

Checks whether at least one expected evidence page is retrieved.

Document Accuracy

Checks whether the expected document is retrieved.

Use these metrics to understand retrieval quality rather than relying on a single accuracy percentage.

Task 8.3 — Implement Answer Metrics

Measure:

Answer Correctness: Whether the generated answer matches the verified expected answer.

Citation Accuracy: Whether the cited document and page actually support the claims.

Refusal Accuracy: Whether the system correctly refuses unanswerable questions.

For numerical questions, compare normalized numerical values while considering units, rounding and reporting periods.

Do not treat a correct-looking answer with an incorrect citation as a successful result.

Task 8.4 — Evaluation Script

Create:

`eval/run_eval.py`

The script should:

1. Load the evaluation questions.
2. Execute retrieval.
3. Compare retrieved evidence against expected evidence.
4. Execute answer generation.
5. Evaluate answers and citations.

6. Calculate aggregate metrics.
7. Save detailed results.

Suggested output:

```
None
Evaluation Summary
------------------
Total Questions: 30

Retrieval:
Recall@5: TBD
MRR: TBD
Document Accuracy: TBD
Page Accuracy: TBD

Answer Quality:
Answer Correctness: TBD
Citation Accuracy: TBD
Refusal Accuracy: TBD

Average Answer Latency: TBD seconds
```

Also save per-question results for error analysis.

Expected Output

A reproducible evaluation framework with at least 30 questions and separate retrieval and answer quality measurements.

Completion Criteria

- 20 provided questions included.
- At least 10 additional questions verified.
- Retrieval metrics implemented.
- Answer metrics implemented.
- Evaluation script runs successfully.
- Final metrics saved.
- Baseline and improved retrieval compared.

PHASE 9 — Error Analysis and Failure Investigation

Estimated Time: 1–2 hours

Objective

Understand where the system fails and why.

Identify the three worst retrieval or answer failures.

For each failure, document:

1. Original question.
2. Expected document and page.
3. Retrieved documents and pages.
4. Actual generated answer.
5. Root cause.
6. Proposed improvement.

Potential root causes:

- PDF extraction failure.
- Table structure lost.
- Incorrect chunk boundaries.
- Relevant document ranked too low.
- Similar figures from different years.
- Incorrect company or period identification.
- Insufficient context for calculation.
- Unsupported LLM-generated claims.

Include these investigations in `EXPERIMENTS.md`.

Expected Output

Three detailed failure investigations with practical next steps.

PHASE 10 — CLI Integration and Final Testing

Estimated Time: 1–2 hours

Objective

Ensure the final implementation follows the assignment's exact interface contract.

Required Commands

1. Ingest

```
None
python main.py ingest <corpus_dir>
```

Expected behavior:

- Read the PDF corpus.
- Load manifest metadata.
- Extract text and tables.
- Generate chunks and embeddings.
- Store them in ChromaDB.
- Skip unchanged filings.
- Update changed filings.

2. Search

```
None
python main.py search --role <role> --k <n> "<question>"
```

Expected behavior:

- Validate the role.
- Apply access restrictions.
- Retrieve top-k chunks.
- Print a JSON list.

3. Ask

```
None
python main.py ask --role <role> "<question>"
```

Expected behavior:

- Retrieve relevant evidence.
- Generate a grounded answer.
- Validate citations.
- Return `answered` or `refused`.
- Print one JSON object.

Additional Testing

Test:

- All four roles.
- All three commands.
- Empty questions.
- Invalid roles.
- Invalid top-k values.
- Missing PDF files.
- Repeated ingestion.
- Changed PDF ingestion.
- Questions involving different years.
- Questions involving multiple companies.
- Unanswerable questions.
- Cross-desk access attempts.

Ensure stdout contains only the required JSON for search and ask. Send diagnostic logs to stderr.

Expected Output

All required commands work with the exact argument structure expected by the evaluator.

PHASE 11 — Documentation and Submission

Estimated Time: 2 hours

Complete all required documentation.

11.1 README.md

Maximum two pages.

Include:

- Project overview.
- Architecture.
- Technology stack.
- Installation instructions.
- Environment configuration.
- Ingestion instructions.
- Search and ask examples.
- Final evaluation results.
- Known limitations.

11.2 DECISIONS.md

Maintain this file throughout development.

Target: 8–12 meaningful decisions.

Each entry should include:

- Decision.
- Options considered.
- Evidence.
- Chosen approach.
- Trade-offs.
- What evidence would change the decision.

Potential decision topics:

1. PDF parser selection.
2. Table extraction strategy.
3. Chunking strategy.
4. Embedding model.
5. Vector database.
6. Chunk ID and re-ingestion strategy.
7. Access-control implementation.
8. Baseline retrieval approach.
9. Hybrid retrieval decision.
10. LLM selection.
11. Refusal strategy.
12. Evaluation metrics.

Only document decisions actually made and supported by your observations.

11.3 EXPERIMENTS.md

Include:

- Baseline experiment.
- Every retrieval experiment.
- Hypothesis.
- Configuration changes.
- Evaluation metrics.
- Final decision.
- Three worst failures.

Include unsuccessful experiments.

11.4 AI_NOTES.md

Write one short paragraph explaining:

- Which AI tools were used.
- How they helped with development.
- One instance where an AI tool gave incorrect or misleading guidance.
- How the mistake was identified and corrected.

Use an actual example from your development process.

11.5 Git History

Commit changes throughout development.

Suggested commit sequence:

```
None
1. Initialize project structure
2. Add corpus inspection and analysis
3. Implement page-wise PDF extraction
4. Add financial table extraction
5. Implement page-aware chunking
6. Add embedding generation
7. Integrate ChromaDB
8. Implement baseline vector retrieval
9. Add role-based access filtering
```

```
10. Implement grounded answer generation
11. Add refusal and citation validation
12. Add evaluation framework
13. Implement retrieval experiments
14. Add failure analysis
15. Finalize documentation and tests
```

These are suggested milestones, not commits to create artificially. Commit when the corresponding work is actually complete.

11.6 Demo Video

Maximum 5 minutes.

Suggested structure:

Duration	Content
0–1 minute	Demonstrate ingestion, search and ask
1–2 minutes	Explain architecture and access control
2–3 minutes	Explain baseline and retrieval improvements
3–4 minutes	Show evaluation results and failure analysis
4–5 minutes	Discuss the least certain decision and what you learned from experiments

The assignment mentions a one-minute demo and allows the remaining time for reasoning. Prioritize explaining your engineering decisions.

---

4. Final Project Acceptance Checklist

Before submission, verify every requirement.

Requirement	Expected Result	Status

Dataset	All 119 PDFs processed	Pending
Manifest	Metadata correctly mapped	Pending
PDF extraction	Page-wise extraction works	Pending
Table extraction	Financial tables searchable	Pending
Chunking	Page-aware chunks generated	Pending
Embeddings	Embeddings generated locally	Pending
Vector database	ChromaDB populated	Pending
Idempotency	No duplicate chunks	Pending
Baseline	Working vector search	Pending
Access control	Four roles correctly restricted	Pending
Retrieval improvement	Measurable improvement over baseline	Pending
Ask command	Grounded answers with citations	Pending
Refusal	Unsupported questions refused	Pending
Calculations	Inputs and formulas shown	Pending
Evaluation	At least 30 questions	Pending
Failure analysis	Three worst failures documented	Pending

Decisions	8–12 meaningful entries	Pending
Experiments	All experiments recorded	Pending
AI Notes	One paragraph completed	Pending
CLI	Exact interface contract supported	Pending
Performance	Answer latency measured against 5-second budget	Pending
Git	Full commit history maintained	Pending
README	Maximum two pages	Pending
Demo	Video within five minutes	Pending

---

5. Three-Day Execution Schedule

Day 1 — Data Understanding and Baseline

Goal: Get the corpus into a searchable vector database.

Tasks:

1. Inspect all dataset files.
2. Analyze manifest and development questions.
3. Inspect representative PDFs.
4. Set up the Python project.
5. Implement PDF text extraction.
6. Implement financial table extraction.
7. Implement page-aware chunking.
8. Generate embeddings.
9. Integrate ChromaDB.
10. Implement baseline search.

11. Record initial retrieval metrics.

End-of-day expected result:

A functional ingestion pipeline and baseline vector search over the provided filings.

Day 2 — Access Control and Grounded Answers

Goal: Complete the end-to-end retrieval and question-answering system.

Tasks:

1. Implement role-based metadata filtering.
2. Test cross-desk restrictions.
3. Implement the ask command.
4. Integrate a local LLM.
5. Implement citation validation.
6. Implement refusal handling.
7. Add calculation support.
8. Create the initial evaluation dataset.
9. Implement retrieval and answer metrics.

End-of-day expected result:

A working CLI system supporting ingestion, search and grounded question answering.

Day 3 — Optimization and Submission

Goal: Demonstrate measurable improvement and prepare the submission.

Tasks:

1. Experiment with chunk sizes.
2. Evaluate hybrid search.
3. Test table-aware retrieval.
4. Compare retrieval metrics.
5. Select the best-performing configuration.
6. Investigate three worst failures.
7. Measure answer latency.
8. Complete documentation.
9. Run final tests.
10. Record the demo video.
11. Verify repository and Git history.

End-of-day expected result:

A complete, tested and documented assignment with measurable retrieval improvement.

---

6. Final Engineering Principles

Throughout development, follow these rules:

1. Do not optimize before measuring the baseline.
2. Do not assume a retrieval technique is better without experimental evidence.
3. Preserve the relationship between financial figures, labels, reporting periods and pages.
4. Enforce access control before returning retrieval results.
5. Never let the LLM invent missing financial information.
6. Treat citations as part of answer correctness.
7. Keep all components locally runnable without paid accounts.
8. Measure performance instead of making unsupported claims.
9. Document failures and unsuccessful experiments honestly.
10. Prioritize a reliable, understandable system over unnecessary complexity.

Final Success Criteria:

The assignment is complete when the system can ingest all provided filings, retrieve relevant evidence under the correct access restrictions, generate grounded answers with page citations, refuse unsupported questions, and demonstrate measurable retrieval improvement over its initial baseline.